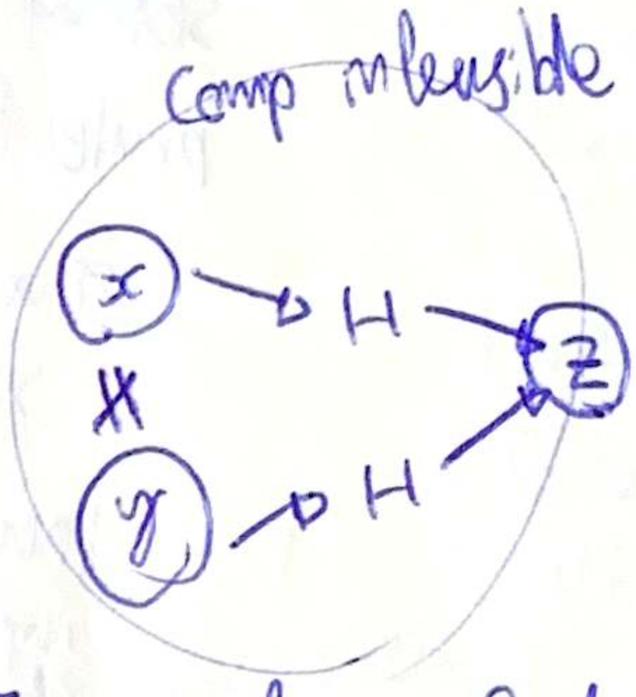
Cryptographic Hash Functions

- A hash function efficiently maps input string of any size to fixed output string size (256-bit)
- CHFs are relevant because cryptocurr systems rely purely on cryptography to implement integrity in currency, no intermediary to resolve dispute.
  - "cryptography" in BTC
    - dig. signatures
    - hash fcnes.

Properties of CHFs s.t. CHF is crypto-secure

1) Collision Resistance

A CHF H is collision-resistant if
- comp infeasible to find
- two distinct inputs, $x \neq y$,
- that produce $H(x) = H(y)$



Because inputs > outputs, collisions do exist BUT comp infeas. to find.

→ Practical Application: Message Digest / Checksum
- Large file on device f
- Compute hash → f'
- Upload to storage online → F
- Download F in future
- Compute hash → F'
- if f' = F' then f = F

→ Practical Application: Commitment Scheme
- Digital version of: store a commitment in an envelope, with a seal on envelope. If commitment changes... seal breaks.
- Commitment = msg
- Seal = msg_hash.

CHF

Properties of CHF s.t. crypto-secure

2) Hiding Property

CHFs are one-way functions, you can't determine input from a hash.

i.e., $H(x) = y$

$\nexists\ H^{-1}$ s.t. $H^{-1}(y) = x$

- Note: A catch is that we must choose x from set of inputs that is large enough s.t. one cannot brute force $H(x_i) = y_i$ and observe

H:
$x_1 \to y_1$
$x_2 \to y_2$.

e.g. x_1 = "heads" y_1 = <256-bit-number>
x_2 = "tails" y_2 = <256-bit-number>

then I compute $H(x_i)\ \forall [x_1, x_2]$ and match the inputs and outputs. Can be done because $[x_1, x_2]$ does not fit HIGH MIN-ENTROPY

↳ input set is so vast and randomly distributed that no particular value is likely

→ Practical Application: Bit Commitment Scheme.

If my message can only be 'I love you'/'I hate you' then we don't have high min entropy.

concat ∴ NONCE = random number used once

$H(x_i \| \text{nonce}) = y_i$

↳ ∴ high min-entropy

∴ Hiding ✓

& ∴ Binding: H is collision resistant ∴ not possible for two candidates to create same hash ∴ cannot change commitments to commit secretly.

CHF

- Properties of CHF s.t. crypto scheme } not hard requirement but is for proof of work
  3) Puzzle Friendliness ≈ must brute force x given y in H(x) = y

A CHF is puzzle friendly iff. there is no mathematical optimisation to force H to produce specific output y in time less than 2^n where $y \in [0; 2^n - 1]$

Formally: $H(k \| x) = y$
y is chosen
k is chosen high min-entropy
computes to choose x in less than $O(2^n)$ ~~runtime~~ time-complexity
brute force

Practical Application: $H(id \| x) = y \in Y$
id = puzzle-id
Y = target set
find x that produces y to solve puzzle } Proof of Work, Bitcoin Mining

Difficulty: size of Y set
- Y full output set → 0 difficulty
- Y is 1 number in output set → 100 difficulty

typically $y \leq Y$ → this operator is used.

HashFuncBTC: SHA-256

$H(id \| x) = y \leq Y$
↳ block.

■ CHF: SHA256 & Merkle Damgard

→ Hash func producing 256 bit output: $[0; 2^{256}-1]$

→ Accepts any input size

→ Uses Merkle-Damgard transform

↳ compression function

↳ allows fixed-input hash funcs to process arbitrary inputs securely & reliably.

↳ as long as compression func is collision resistant,

so will the hash func

i.e. Merkle Damgard → Col. Res

∴ SHA256 → Col. Res.

How: C: comp func.

- $C(x) = y$
- $len(x) = m$ (fixed) 768
- $len(y) = n$ (fixed) 256

→ Now say we want to Hash file F of arb length.

768 256 = 512

F ⟵ divided into blocks of len $(m-n)$

b_i b_i

Each 512 block b_i → $C(x_i = b_i \,||\, y_{i-1}) = y_i$

256

concat w/ prev output y_{i-1}

and processed as 768 block

x_i	b_i	y_{i-1}	y_i
768 len	512 len	256 len	256 len

→ If $i=1$ then $y_0 = IV$ (init vector)

y_i → hash. → hash pointer to

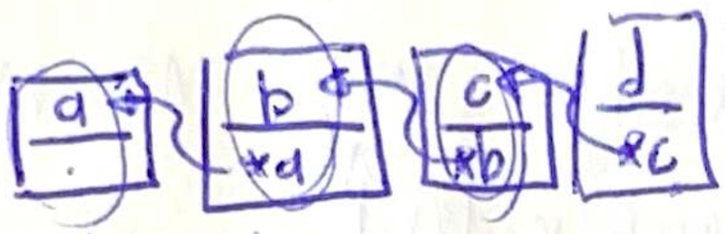
→ The final y_n output → $H(F) = y_n$

→ $H(F)$ far more efficient when relying on $C(x)$ func

BlockChain = LinkedList w/ Hash Pointers

→ Hash pointers:
- points to information at address
AND - tamper evidence of information

→ Take a LL



→ Replace ptr w/ hash ptr (hash ~~computed~~ on block including its ptr) NB.
= ~~tamper~~ evident log
= blockchain.

* doubly-linked also stores child ref.
* relevant for fwd + bwd traversal.

⊛ ... 1 bit changes in a, b, c, d .... whole data structure invalid all the way to the ~~head~~. genesis block.

⊛ only need to remember (consensus) latest hash at the head.
to verify state.

⊛ if one remembers a truthful set of state transitions, one can reconstruct truth from genesis block.

⊛ in a dlb network of signers, conduct a puzzle search for each state transition to grant signing rights
= bitcoin.